

## **MILESTONE 2: Core Functionalities Development**

### **Activity 2.1. : Module Implementation**

#### **1. Explanation Module**

EduGenie utilizes the LaMini-Flan-T5 model, a lightweight yet powerful generative AI, to deliver educational content in a simplified and highly readable manner. This model is specifically fine-tuned to provide concise, context-aware responses that break down complex topics into easily understandable language. By focusing on clarity and brevity, EduGenie ensures that learners with minimal background knowledge or technical experience can grasp essential concepts without feeling overwhelmed. This makes it particularly valuable for beginners, school students, or self-learners seeking straightforward explanations. The integration of LaMini-Flan-T5 empowers EduGenie to act as a reliable and accessible study companion for foundational learning across subjects.

#### **Explanation module logic**

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

import torch


# Load the improved model for explanations

explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")


def explain_topic(topic: str) -> str:

    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."

    inputs = explain_tokenizer(input_text, return_tensors="pt")


    outputs = explain_model.generate(

        **inputs,

        max_new_tokens=150,

        temperature=0.7,

        top_k=50,
```

```

        top_p=0.95,

        do_sample=True
    )

    explanation = explain_tokenizer.decode(

        outputs[0],

        skip_special_tokens=True
    )

    return explanation

```

## 2. QnA Module

EduGenie is powered by Gemini 1.5 Pro, enabling it to handle a wide range of general knowledge and academic question-answering tasks with precision. Its advanced understanding and contextual capabilities make it an effective tool for learners seeking accurate, AI-driven assistance across diverse educational topics and subjects.

### Q/A Module

```

def answer_question_with_gemini(question: str) -> str:

    try:

        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")

        response = model.generate_content(question)

        return response.text.strip()

    except Exception as e:

        return f"⚠ Error in QnA: {e}"

```

## 3. Quiz Module

EduGenie generates three multiple-choice questions (MCQs) from a given passage, each containing four carefully crafted options. It utilizes the Gemini model to comprehend the context and meaning of the passage, ensuring that the questions are relevant and the distractors are plausible. The final output is structured in JSON format for easy integration.

### Quiz Module Logic

```

def clean_json_block(text):

```

```
# Remove Markdown ```json code fences
```

```
return re.sub(r"```(?:json)?\n(?:.*?)```", r"\1", text, flags=re.DOTALL).strip()
```

```
def generate_quiz(text: str) -> list:
```

```
    try:
```

```
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
```

```
        prompt = f"""
```

```
You are a quiz generator.
```

From the following passage, create 3 multiple-choice questions. Each question should include:

- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **\*\*valid JSON\*\***, like this:

```
[  
  
  {  
  
    "question": "What is ...?",  
  
    "options": ["A", "B", "C", "D"],  
  
    "answer": "A"  
  
  }  
  
]
```

Passage:

```
{text}
```

"""

```
response = model.generate_content(prompt)
```

```
quiz_text = response.text.strip()
```

```
# ✍ Clean markdown code blocks if any
```

```
cleaned_text = clean_json_block(quiz_text)
```

It sends a structured prompt to the model, expecting a valid JSON response with each question containing four options and a correct answer. To ensure proper parsing, it cleans any Markdown code blocks using the `clean_json_block` function. The cleaned response is then parsed into a Python list. If an error occurs during generation or parsing, it returns a detailed error message for debugging.

#### 4. Summary Module

This feature leverages Gemini's generative capabilities to summarize long paragraphs into concise, easy-to-understand versions, making it ideal for quick revision. It ensures the core information is retained while eliminating redundancy, helping learners absorb key points efficiently without losing clarity or context.

##### **Summary Module**

```
def summarize_text(text: str) -> str:
```

```
    try:
```

```
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
```

```
        prompt = f"Summarize the following text in simple language:\n\n{text}"
```

```
        response = model.generate_content(prompt)
```

```
        return response.text.strip()
```

```
    except Exception as e:
```

```
        return f"⚠ Error in Summary: {e}"
```

#### 5. Learning Path Module

The `get_learning_recommendations` function uses Google Gemini to generate a personalized, structured learning path for any given topic. By crafting a detailed prompt, it instructs the AI to suggest beginner to advanced concepts, organized by difficulty, and supported with useful

resources such as videos, articles, or books. It ensures adaptability to the learner's level. The function safely handles API responses and errors, providing either the generated text or a helpful error message if the response is invalid or incomplete.

### Learning Path Module

```
def get_learning_recommendations(topic):

    prompt = f"""

    You are an AI tutor. The student wants to learn about: {topic}.

    Suggest a structured and adaptive learning path, including key topics, order of learning, and
    resources (if

    Include beginner, intermediate, and advanced levels if needed.

    """

    try:

        response = model.generate_content(prompt)

        print("🔗 Gemini raw response:", response)

        if hasattr(response, "text"):

            return response.text

        elif hasattr(response, "parts") and response.parts:

            return response.parts[0].text

        else:

            return "❌ Could not extract content from Gemini response."

    except Exception as e:

        import traceback

        traceback.print_exc()

        return f"Error occurred: {str(e)}"
```

### Activity 2.2 : Backend API with FastAPI

- Defined RESTful endpoints for each module:
  - /qa
  - /explain
  - /quiz
  - /summarize
  - /learn/recommendations
- Connected each endpoint to a respective module logic

## Endpoints of API

# Q&A - GET API using Gemini

```
@app.get("/qa")
```

```
async def answer_question(question: str = Query(...)):
```

```
    answer = answer_question_with_gemini(question)
```

```
    return {"answer": answer}
```

# Explanation - POST API

```
@app.post("/explain/")
```

```
async def explain_api(request: Request):
```

```
    data = await request.json()
```

```
    topic = data.get("topic")
```

```
    if not topic:
```

```
        return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
```

```
    explanation = explain_topic(topic)
```

```
    return {"topic": topic, "explanation": explanation}
```

# Summarization - POST API

```
@app.post("/summarize/")
```

```
async def summarize_api(request: Request):
```

```
    data = await request.json()
```

```

text = data.get("text")

if not text:

    return JsonResponse(content={"error": "Please provide text to summarize."},
status_code=400)

summary = summarize_text(text)

return {"summary": summary}

```

### **Endpoints of API**

# Quiz Generation - POST API

```
@app.post("/quiz")
```

```
async def quiz_api(request: Request):
```

```
    data = await request.json()
```

```
    text = data.get("text")
```

```
    if not text:
```

```
        return JsonResponse(content={"error": "Please provide text for quiz."}, status_code=400)
```

```
    quiz = generate_quiz(text)
```

```
    print("Generated quiz:", quiz)
```

```
    return JsonResponse(content={"quiz": quiz})
```

# Learning Recommendations - GET API

```
@app.get("/learning-recommendations")
```

```
async def learning_recommendation_api(topic: str = Query(...)):
```

```
    recommendation = get_learning_recommendations(topic)
```

```
    return {"topic": topic, "recommendation": recommendation}
```